

Basic Concepts

Advanced Programming
Brent van Bladel



Compiled vs Interpreted Languages

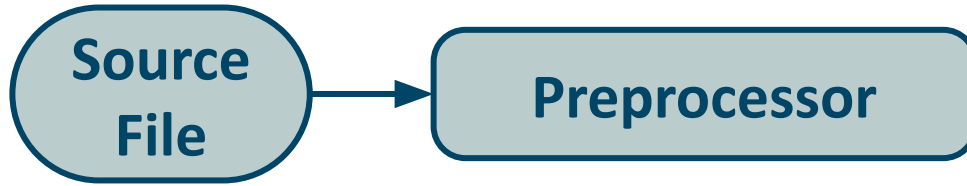
Compiled

Source code is converted to executable machine code before runtime.

Interpreted

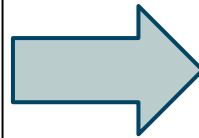
Source code is converted to executable machine code at runtime.

C++ Compiler Phases: Preprocessor



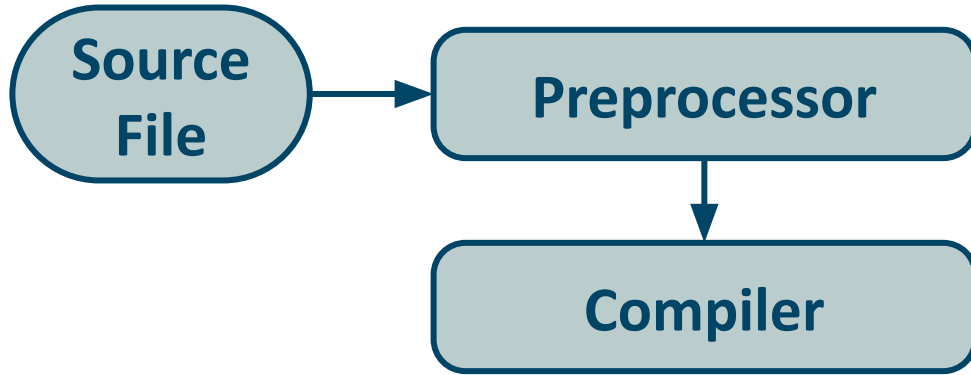
Source code is preprocessed using **macros** (start with #).

```
1. #define MIN(a,b) ((a < b) ? a : b)
2.
3. int main(){
4.     std::cout << "The minimum is";
5.     std::cout << MIN(37, 42);
6. }
```



```
1. int main(){
2.     std::cout << "The minimum is";
3.     std::cout << ((37 < 42) ? 37 : 42);
4. }
```

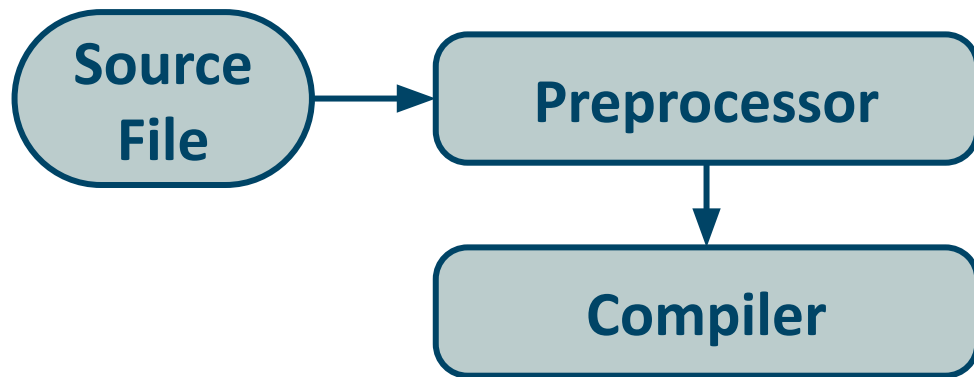
C++ Compiler Phases: Builder



Preprocessed source code is converted to machine code, typically object files.

- One object file for each source file is created.
- Optimization is performed

C++ Compiler Phases: Building

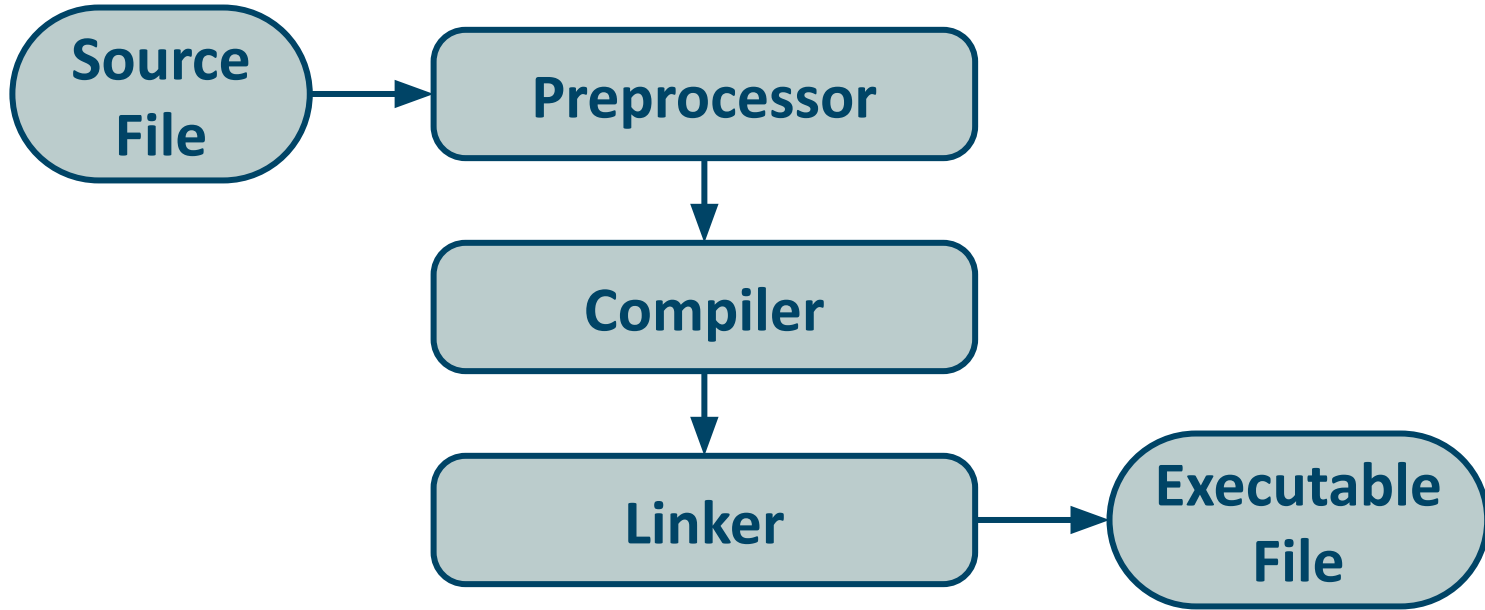


```
1. int main(){  
2.     std::cout << "The minimum is";  
3.     std::cout << ((37 < 42) ? 37 : 42);  
4. }
```

```
.LC0:  
    .string "The minimum is"  
main:  
    push    rbp  
    mov     rbp, rsp  
    mov     esi, OFFSET FLAT:.LC0  
    mov     edi, OFFSET FLAT:std::cout  
    call    std::basic_ostream<...>& std:::  
    mov     esi, 37  
    mov     edi, OFFSET FLAT:std::cout  
    call    std::basic_ostream<...>::opera  
    mov     eax, 0  
    pop     rbp  
    ret
```

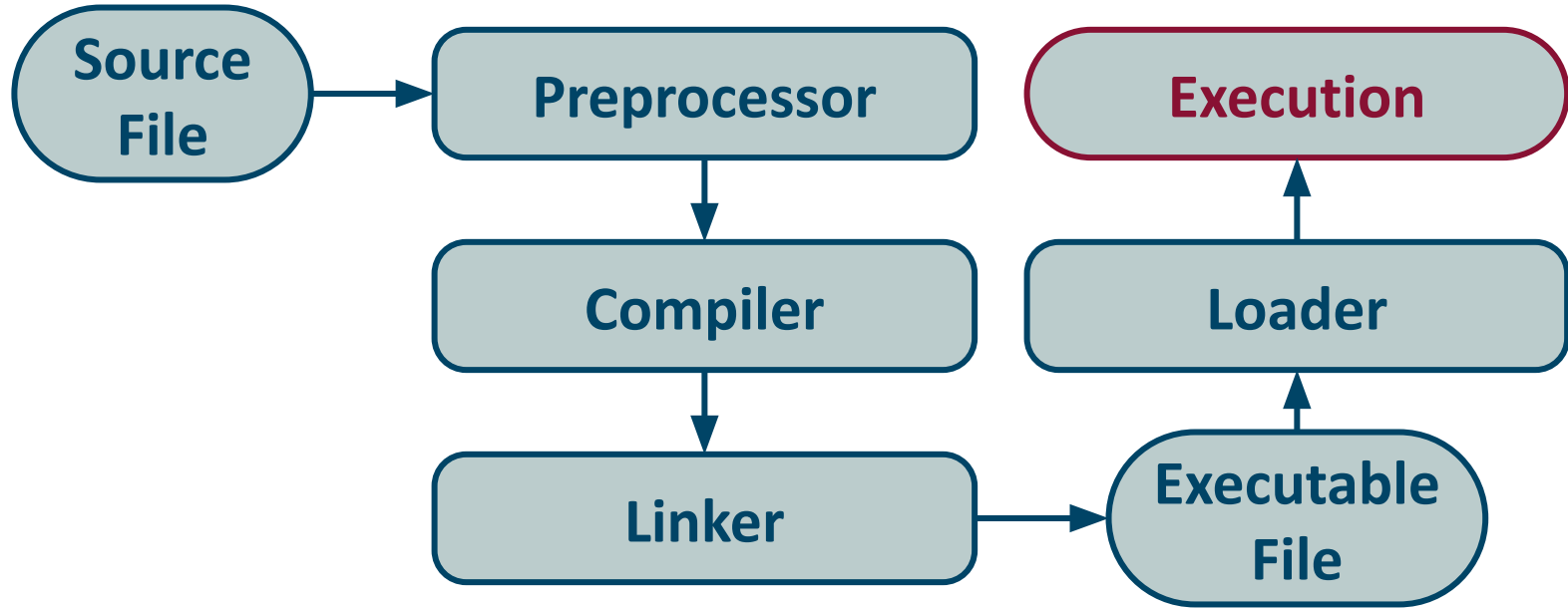
Optimization:
calculated at compile time

C++ Compiler Phases: Linker



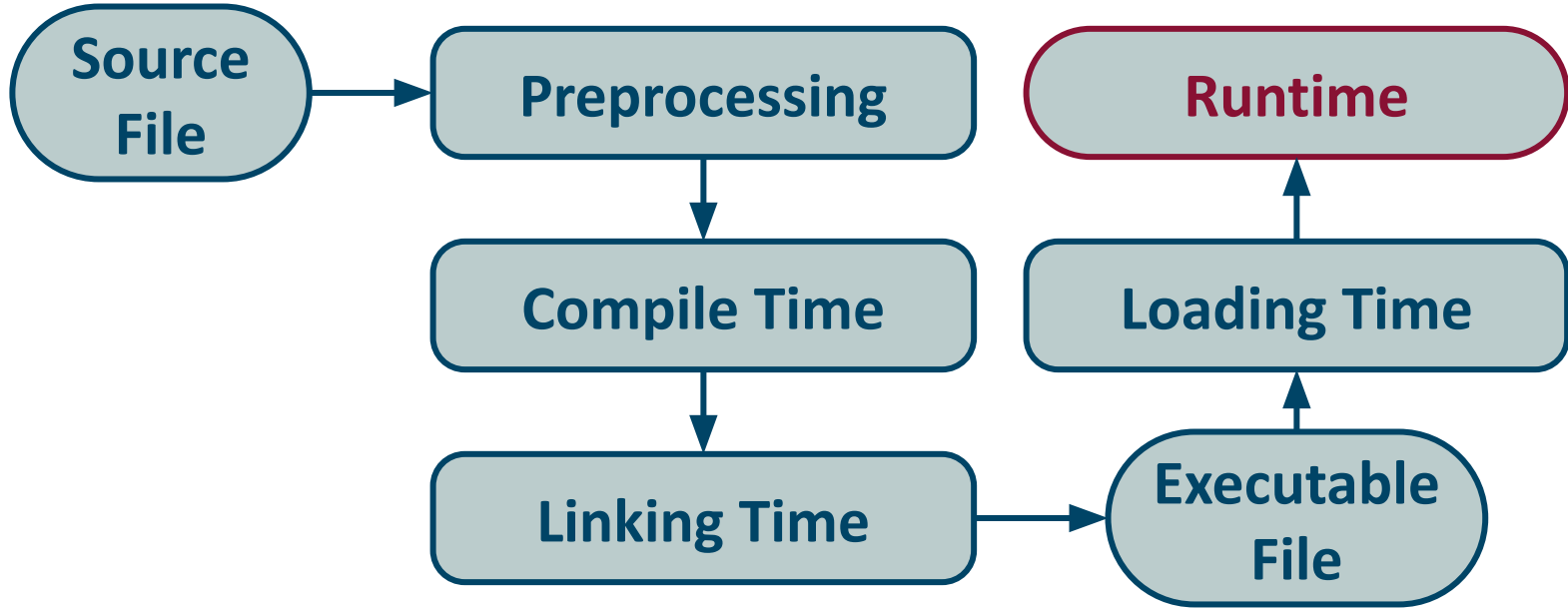
Object files are linked to create one executable file.

C++ Compiler Phases: Loader



The executable code is loaded into memory, after which it can be run.

C++ Timings



Binding

Binding is the act of associating properties with names.

Note that different properties are associated at different times.

Binding Time

```
1.  int x;  
2.  
3.  int main(){  
4.      int y = 42;  
5.      x = doSomething(y);  
6.  }
```

- When is the type of *x* bound? **Compile time**
- When is the reference to *doSomething** bound? **Linking time**
- When is the memory location for *x* bound? **Loading time**
- When is the memory location for *y* bound? **Runtime**
- When is the value of *x* bound? **Runtime**

Static vs Dynamic Binding

Static Binding occurs before runtime and remains unchanged during the program execution.

Dynamic Binding occurs during program execution and can change during the program execution.

Binding Time

```
1.  int x;  
2.  
3.  int main(){  
4.      int y = 42;  
5.      x = doSomething(y);  
6.  }
```

Static Binding

- When is the type of `x` bound? **Compile time**
- When is the reference to `do_something*` bound? **Linking time**
- When is the memory location for `x` bound? **Loading time**
- When is the memory location for `y` bound? **Runtime**
- When is the value of `x` bound? **Runtime**

Binding Time

```
1.  int x;  
2.  
3.  int main(){  
4.      int y = 42;  
5.      x = doSomething(y);  
6.  }
```

- When is the type of `x` bound? **Compile time**
- When is the reference to `do_something` bound? **Linking time**
- When is the memory location for `x` bound? **Loading time**

Dynamic Binding

- When is the memory location for `y` bound? **Runtime**
- When is the value of `x` bound? **Runtime**

Static vs Dynamic Semantics

Static Property

- Applies to the source code, not to a execution.
- Can be determined from source code.

Dynamic Property

- Occurs during program execution.
- Can be inferred from the execution of a program.

Static vs Dynamic Types

Static Type

The type of an expression, which type results from analysis of the program without considering execution semantics.

Dynamic Type

The type of the most derived object to which the expression refers.

Note that every variable has both a static type and a dynamic type.

Static vs Dynamic Types Example

```
1. struct Base {};  
2. struct Derived : public Base {};  
3.  
4. void doSomething(Base& b) {...}  
5.  
6. int main(int argc, char* argv){  
7.     Base base;  
8.     Derived derived;  
9.     doSomething(argc ? base : derived);  
10. }
```

Variable *base*:

- **Static type:** Base
- **Dynamic type:** Base

Static vs Dynamic Types Example

```
1. struct Base {};  
2. struct Derived : public Base {};  
3.  
4. void doSomething(Base& b) {...}  
5.  
6. int main(int argc, char* argv){  
7.     Base base;  
8.     Derived derived;  
9.     doSomething(argc ? base : derived);  
10. }
```

Variable *derived*:

- **Static type:** Derived
- **Dynamic type:** Derived

Static vs Dynamic Types Example

```
1. struct Base {};  
2. struct Derived : public Base {};  
3.  
4. void doSomething(Base& b) {...}  
5.  
6. int main(int argc, char* argv){  
7.     Base base;  
8.     Derived derived;  
9.     doSomething(argc ? base : derived);  
10. }
```

Variable *b*:

- **Static type:** Base
- **Dynamic type:** either Base or Derived

Static vs Dynamic Types Example

```
1. struct Base {};  
2. struct Derived : public Base {};  
3.  
4. void doSomething(Base& b) {...}  
5.  
6. int main(int argc, char* argv){  
7.     Base base;  
8.     Derived derived;  
9.     doSomething(base);  
10.    doSomething(derived);  
11. }
```

Variable *b*:

- **Static type:** Base
- **Dynamic type:** Base, then Derived

Declarations & Definitions

A **declaration** introduces an identifier and binds its static type.

It is what the compiler needs to accept references to that identifier.

A **definition** instantiates/implements an identifier.

It's what the linker needs in order to link references to those entities.

```
1.  extern int bar;  
2.  extern int g(int, int);  
3.  double f(int, double);  
4.  class Foo;
```

```
1.  int bar;  
2.  int g(int a, int b) {  
3.      return a + b; }  
4.  double f(int a, double b) {  
5.      return a * b; }  
6.  class Foo {  
7.      };
```

Note that a definition can also be a declaration.

Declarations & Definitions

An identifier can be **declared** as often as you want.

```
1. int foo();  
2. int foo();  
3.  
4. int foo() { return 42; };
```

An identifier must be **defined** exactly once!

```
1. int foo() { return 42; };  
2.  
3. int foo() { return 42; };  
4. // linker error
```

One Definition Rule

- No translation unit shall contain more than one definition of any variable, function, class type, enumeration type or template.
- Every program shall contain exactly one definition of every non-inline function or variable that is used.
- Exactly one definition of a class is required if any translation unit where the class is used, requires it to be complete.

No definition: unknown what to do.

Multiple definitions: unknown which to choose.